

SchemaSense AI

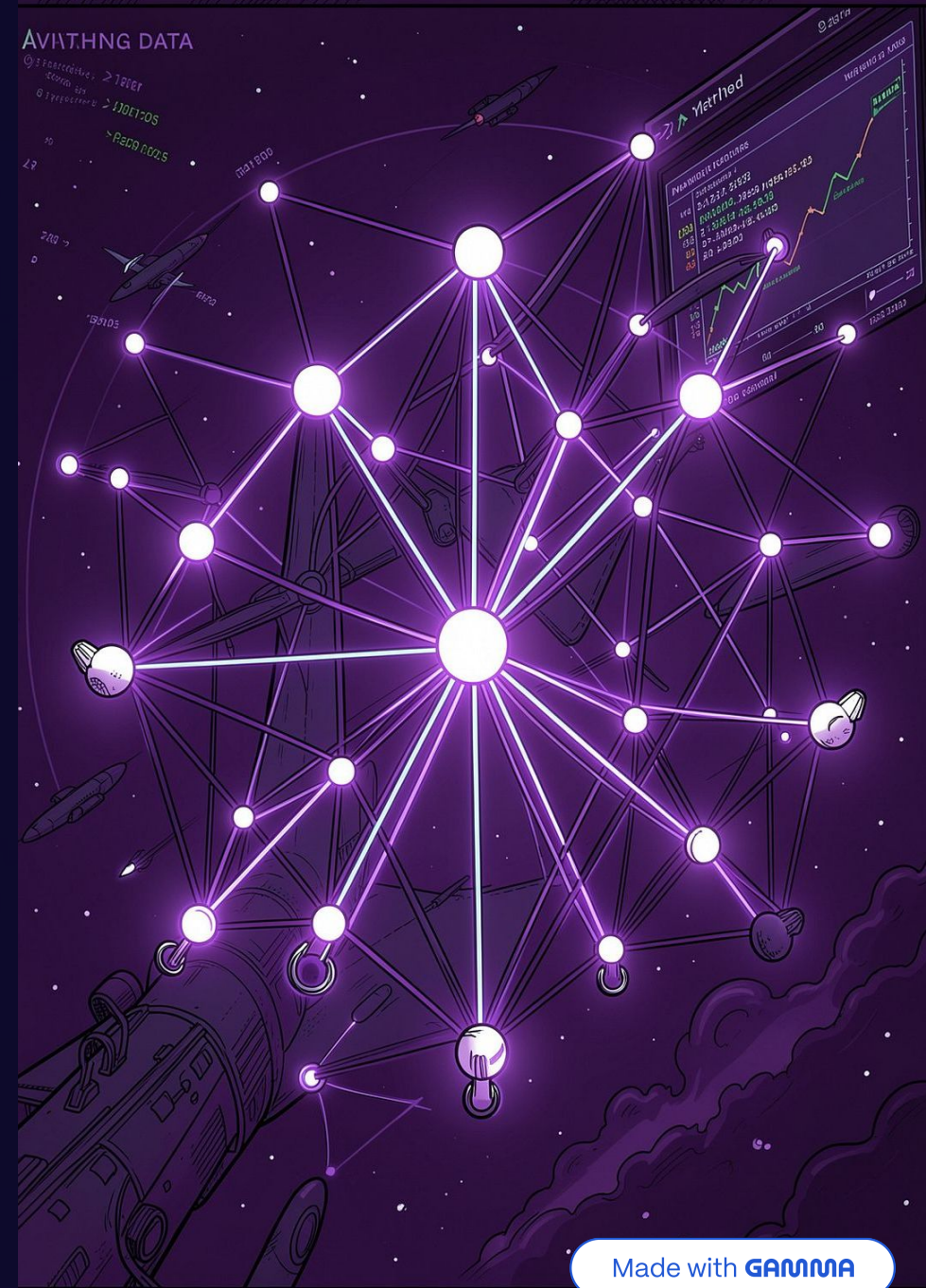
Ask Your Warehouse. See the Relationships.

AI-powered metadata discovery and verified ER diagrams

MAIMUNAH ISKHANDER (2777845)

SEPTEMBER 23, 2026

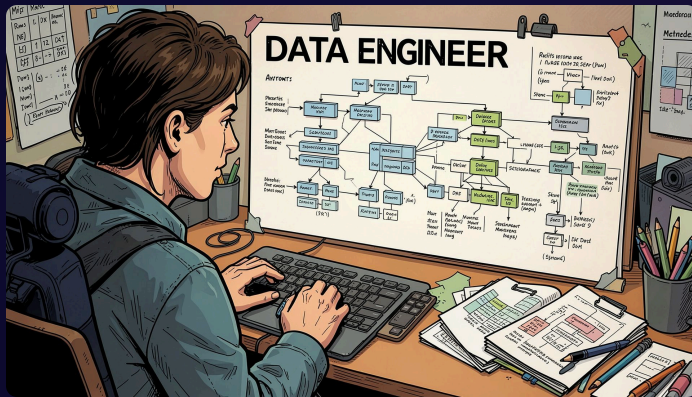
Generative AI E2 Capstone



Made with GAMMA

USE CASE

I Know the Tables Exist. But How Do I Join Them?



The Problem

New team members often need to inspect schemas, columns, and relationships manually before they can write SQL confidently. Without a clear ER diagram, identifying the right joins can be time-consuming and error-prone.

Expected Value

- Faster table discovery
- Clearer PK/FK relationships
- Easier SQL development

10

Tables

3

Schemas

11

FK Links

Users: Data engineers, analysts, BI developers, and new joiners · **Synthetic data only**

Two Simple Paths

1

Index Metadata

Synthetic JSON → Validate & Chunk → OpenAI Embeddings
→ Pinecone

2

Answer User Questions

Flask UI → Input Guardrails → LangGraph Tools → Answer
or ER Diagram

Core Stack

gpt-5-nano · OpenAI embeddings · Pinecone · LangGraph ·
Flask · Python



Exact PK and FK checks verify every relationship
shown in an ER diagram.

How SchemaSense Finds the Right Tables

01

Prepare

One document per table

02

Embed

text-embedding-3-small, 1536 dimensions

03

Store

Pinecone namespace `synthetic-v1`

04

Retrieve

Top 5 matches with optional schema filter

05

Verify

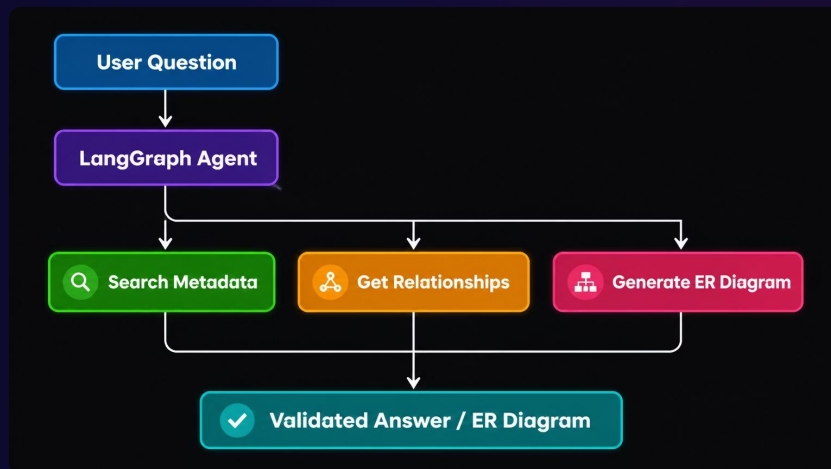
Citations + exact PK/FK relationship checks

The answer cites fully qualified tables. User questions are combined with retrieved metadata before generation.

One Agent, Three Tools

How the Agent Works

- Chooses among 3 controlled tools at each step
- Records each tool in a visible trace



Agent Guardrails

- Maximum 4 tool calls per request
- Validate tool outputs before use
- Reject unknown tables and relationships
- Generates ER diagrams only from verified metadata
- Limits diagrams to relevant tables

search_metadata

Find relevant table metadata

get_relationships

Retrieve verified PK and FK relationships

generate_er_diagram

Render a validated Mermaid ER diagram

GUARDRAILS

Prompt Engineering and Guardrails

Role-based Prompting

The agent is given a structured system prompt that defines its identity, capabilities, and constraints before any user interaction begins.

Identity

You are a data modeling assistant. You help users understand database schemas by analyzing table metadata and relationships.

Capabilities

You can search for table metadata, retrieve primary and foreign key relationships, and generate validated ER diagrams.

Constraints

You operate only on verified schema data. You never invent or assume tables, columns, keys, or relationships.

System rule: Use tools before factual claims. Never invent schemas, columns, keys, or relationships.

Agent Guardrails

1

Ground before claiming

Always retrieve metadata with `search_metadata` before making any factual statement about a table or column.

2

Block unsafe inputs

Reject prompt injection attempts, requests for secrets, credentials, or system internals.

3

Enforce schema safety

Reject unknown tables and unverified relationships. Only operate on data returned by tools.

4

Control resources

Limit tool calls to four per request. Keep ER diagrams scoped to the tables explicitly requested.

DEMO

Verified ER Diagram and Answers

DATABASE METADATA COPILOT

SchemaSense AI

Explore synthetic Redshift-style metadata and create verified ER diagrams.

Model: **gpt-5-nano**
NameSpace: **synthetic-v1**

Try asking

Which table stores aircraft details?

How are passengers connected to flights?

Create an ER diagram for flight bookings.

Create an ER diagram for baggage and ULD operations.

Synthetic metadata only. No production or customer data.

Clear conversation

RAG + ONE LANGGRAPH AGENT

Ask about your warehouse structure

AI: Here is the verified ER diagram. The diagram is rendered below using validated metadata.

Verified ER diagram

Ask about schemas, tables, columns, keys, or relationships

User prompt: *Generate ER diagram for entire warehouse*

This is the complete warehouse schema with all 10 tables and 11 FK relationships verified — every table, column, primary key, and foreign key was retrieved and confirmed via tool calls before the diagram was generated.

DEMO

Verified Answer with Citations

AI

The aircraft master data is stored in the aviation.aircraft table.

Key details:

- aircraft_id: PRIMARY KEY
- registration_number
- aircraft_type
- seat_capacity
- active_flag

Related link: aviation.flights.aircraft_id references aviation.aircraft.aircraft_id (to assign an aircraft to flight instances).

Sources: aviation.aircraft; aviation.flights

▼ Retrieved sources

- aviation.aircraft (0.377)
- aviation.flights (0.279)
- aviation.airports (0.255)

User prompt: *Which table stores aircraft details?*

Verified answer with citations — retrieved sources ranked by relevance score.

Agent Tool Trace

▼ Agent tool trace

- `search_metadata`: Retrieved 5 table(s): `operations.uld`, `operations.baggage`, `operations.uld_assignment`, `operations.flight_status`, `aviation.aircraft`.
- `generate_er_diagram`: Generated a validated ER diagram for 5 table(s).

User prompt: *Generate ER diagram for baggage and ULD operations*

This screenshot shows the agent's tool trace — the step-by-step sequence of tool calls the agent made to retrieve metadata and generate the ER diagram, demonstrating how the system verified all relationships before rendering the diagram.

Results, Outcomes and Challenges

10

Table Documents

Ingested into the vector store

11

FK Links Verified

Declared relationships confirmed

3

Diagram Scopes

Focused and full warehouse ER demonstrated

What Worked

- Aircraft questions returned the correct table, key columns, and ranked sources
- ER diagrams rendered accurately from declared relationships

Challenges and Fixes

- **Model access:** Corrected model configuration and environment precedence
- **Agent flow:** Added deterministic routing for ER diagram requests
- **Browser rendering:** Delayed Mermaid rendering until the page was ready



Future improvement: Scale the solution to support more schemas, tables, and data domains.